

Mercari 알고리즘 기반 가격 제안 예측 모델 및 Keras 희소 다층 퍼셉트론 아키텍처 심층 분석 보고서

1. 서론: 개인 간 거래(C2C) 플랫폼의 가격 책정 난제와 알고리즘적 해결의 필요성

현대의 대규모 전자상거래 생태계, 특히 사용자가 직접 판매자와 구매자의 역할을 동시에 수행하는 개인 간 거래(C2C, Consumer-to-Consumer) 플랫폼에서 가장 해결하기 어렵고 복잡한 과제 중 하나는 '적절한 상품 가격의 산출 및 제안'이다. 상품의 내재적 가치를 정확히 평가하고 이를 화폐 가치로 환산하는 것은 고도의 다차원적인 분석을 요하는 작업이다. 상품의 아주 미세한 세부 사항 차이만으로도 시장에서 형성되는 가격의 변동 폭은 극심하게 나타날 수 있다.¹ 플랫폼 상에 등록된 겔보기에는 유사해 보이는 두 벌의 스웨터가 있을 때, 하나는 고급 브랜드의 한정판이거나 특정 빈티지 요소를 갖추어 335달러에 판매되는 반면, 다른 하나는 일반적인 패스트 패션 브랜드의 중고 물품으로 분류되어 9.99달러에 판매되는 극단적인 비대칭성이 일상적으로 발생한다.¹

이러한 제품 가격 책정의 어려움은 매일 수백만 건의 상품이 거래되는 플랫폼의 규모(Scale)와 결합될 때 기하급수적으로 증폭된다.¹ 특히 의류 카테고리의 경우 특정 디자이너나 브랜드 이름이 가격을 견인하는 가장 강력한 변수이며 계절적 수요 변화와 트렌드에 따라 가격이 요동치는 반면, 스마트폰이나 노트북과 같은 전자기기는 브랜드보다는 제품의 세부 사양(Specs), 연식, 그리고 기기적 결함 여부에 따라 가격이 결정되는 등 카테고리별로 가격을 결정하는 잠재적 메커니즘과 변수의 가중치가 완전히 다르기 때문이다.¹

일본 최대의 커뮤니티 기반 쇼핑 앱인 Mercari(메르카리)는 이러한 정보 비대칭성과 가격 산정의 난맥상에 깊이 직면해 있는 대표적인 플랫폼이다.¹ Mercari의 마켓플레이스는 고도로 개방되어 있어 판매자가 의류, 전자기기, 수집품 등 단일 품목뿐만 아니라, 전혀 연관성이 없어 보이는 여러 물품을 묶은 번들(Bundle) 형태에 이르기까지 상상할 수 있는 거의 모든 종류의 상품을 자유롭게 등록할 수 있다.¹ 이러한 자유도는 플랫폼의 활성화를 이끄는 핵심 동력이지만, 동시에 초보 판매자들이 자신의 상품을 얼마에 올려야 할지 몰라 거래를 포기하거나 비합리적인 가격을 설정하여 시장의 효율성을 저해하는 부작용을 낳기도 한다. 따라서 판매자가 상품의 사진, 이름, 카테고리, 상태 등을 입력했을 때 시스템이 즉각적으로 합리적인 '가격 제안(Pricing Suggestions)'을 제공하는 알고리즘을 구축하는 것은 거래 성사율을 극대화하고 사용자 경험을 혁신하기 위한 필수 불가결한 요소이다.¹

본 보고서는 이러한 산업적 배경을 바탕으로 개최된 Mercari 가격 제안 예측 대회(Price Suggestion Challenge)의 문제 상황을 심층적으로 해부하고, 이를 해결하기 위해 구축된 최적화된 딥러닝 코드(main.py)의 동작 방식과 내부 알고리즘 아키텍처를 순차적으로 상세히 분석한다. 외부 데이터의 업로드가 금지된 엄격한 통제 환경 속에서, 대규모 텍스트 데이터를 처리하고 컴퓨팅 리소스의 병목을 극복하여 최고 수준의 예측 성능을 달성한 신경망 설계와

최적화 기법을 구조적으로 조명한다.

2. 대회 환경의 구조적 제약과 데이터 명세 분석

2.1. 인프라 제약: **Kernels Only** 규정과 컴퓨팅 자원의 한계

본 알고리즘 개발의 가장 중대한 제약 사항이자 솔루션의 형태를 결정지은 핵심 요인은 대회가 철저히 "**Kernels Only**(커널 전용)" 방식으로 기획되었다는 점이다.¹ 이는 참가자들이 로컬 머신의 막대한 **GPU** 클러스터나 무제한적인 클라우드 자원을 활용하여 수백 개의 모델을 앙상블하는 전통적인 방식을 원천적으로 차단한다. 주최측은 모든 참가자가 동일한 조건에서 경쟁하도록 **Kaggle** 플랫폼이 제공하는 제한된 클라우드 컨테이너 환경 내에서 데이터 로딩, 전처리, 모델 훈련(**Training**), 그리고 추론(**Prediction**)까지의 모든 파이프라인이 완결되도록 강제했다.¹

커널 환경에 부여된 컴퓨팅 리소스는 단 **4개의 CPU 코어**, **16GB의 시스템 RAM**, **1GB의 임시 스크래치(Scratch)** 및 출력 디스크 공간으로 극히 제한되어 있다.¹ 무엇보다 가장 치명적인 제약은 전체 코드의 실행 시간(**Run-time**)이 **60분**을 초과할 수 없다는 타임아웃 규정이다.¹ 대규모 머신러닝 과제에서 **60분**은 복잡한 자연어 처리(**NLP**) 파이프라인 하나를 구동하기에도 벅찬 시간이다. 모델의 아키텍처가 아무리 정교하더라도 이 **60분**과 **16GB RAM**의 벽을 넘지 못하면 제출 자체가 불가능하므로, 알고리즘의 예측 정확도(**Accuracy**) 못지않게 시간 복잡도(**Time Complexity**)와 공간 복잡도(**Space Complexity**)를 극한으로 억제하는 공학적 설계가 필수적이다.¹

대회 일정은 단계별로 세분화되어 진행되었다.

주요 일정 (2018년 기준)	단계 및 세부 내용	파급 효과 및 제약
2월 7일	참가 등록 및 팀 병합(Team Merger) 마감일	이 날짜 이후 참가자들의 결함이 금지됨. ¹
2월 14일	최종 제출 마감일 및 커널 버전 선택	Stage 2 를 위한 코드가 확정되며, 이후 수정 불가. ¹
2월 15일	Stage 2 평가 기간 시작	선택된 커널이 완전히 새로운 대규모 비공개 데이터셋에서 재실행됨. ¹
2월 21일	Stage 2 종료 및 최종 결과 발표	비공개 테스트 데이터에 대한 검증 완료. ¹

위의 일정표에서 가장 주목해야 할 점은 **2월 15일**을 기점으로 전환되는 데이터 규모의 폭발적인 증가이다.

2.2. 데이터셋 규모의 팽창과 블라인드 테스트(Blind Test) 환경

주최측은 참가자들이 리더보드 점수에 과적합(Overfitting)되는 것을 방지하고 알고리즘의 일반화(Generalization) 능력을 평가하기 위해, 대회를 Stage 1과 Stage 2로 분리하고 데이터의 규모를 극단적으로 변경하는 전략을 취했다.

Stage 1 기간 동안 참가자들에게 참조용으로 제공되는 테스트 데이터 파일(test.tsv 및 sample_submission.csv)은 약 70만(700K) 개의 행으로 구성되어 있다.¹ 이 환경에서 커널의 실행 시간이 60분 이내로 통과되면 Stage 1 리더보드에 점수가 기록된다. 그러나 Stage 2가 시작되면 기존의 70만 개 데이터가 담겨 있던 test.tsv 파일이 무려 350만(3.5M) 개의 행을 포함하는 거대한 비공개 테스트 데이터셋(test_stg2.tsv)으로 은밀하게 교체된다.¹

이러한 5배에 달하는 데이터 볼륨 팽창은 메모리 관리 측면에서 거대한 재앙을 초래할 수 있다. Stage 1에서 10GB의 RAM을 사용하던 파이프라인이 있다면, 데이터가 5배로 늘어나는 순간 여지없이 16GB 메모리 한계를 돌파하여 시스템 충돌(Out of Memory)을 일으키게 된다. 따라서 개발자는 "보이지 않는 데이터(Unseen data)"가 코드의 흐름을 파괴할 수 있음을 미리 인지하고, 예외 처리(Try/Catch blocks)를 통해 에러 발생 시에도 유효한 예측값을 배출할 수 있도록 방어적 프로그래밍을 수행해야 한다.¹ Stage 2로 진입할 때 코드의 파일명이나 실행 구조가 동적으로 변해서는 안 되며, 타임스탬프를 파일명에 삽입하는 등의 행위는 회피되어야 한다.¹

2.3. 입력 변수 명세 및 데이터 누수(Data Leakage) 차단 기법

학습을 위해 제공되는 훈련 데이터(train.tsv)와 추론 대상인 테스트 데이터(test.tsv)는 탭으로 구분된 형식이며, 하나의 상품이 마켓플레이스에 리스팅될 때 필요한 핵심 속성들을 포함하고 있다.

데이터 필드명	데이터 타입	설명 및 특이사항
train_id / test_id	정수형	각 상품 리스팅의 고유 식별자. ¹
name	텍스트 (단문)	판매자가 작성한 상품의 제목. ¹
item_condition_id	범주형 (숫자형)	판매자가 평가한 상품의 상태 코드. ¹
category_name	텍스트 (계층형)	상품이 속한 카테고리 경로 (예: 대/중/소분류). ¹
brand_name	텍스트	상품의 제조사 혹은 브랜드 이름. ¹

shipping	이진형 (Boolean)	배송비 부담 주체. 판매자가 지불하면 1, 구매자가 지불하면 0. ¹
item_description	텍스트 (장문)	상품에 대한 상세한 설명글. ¹
price	연속형 (Float)	타겟 변수. 상품이 실제로 판매된 USD 기준 가격 (테스트 데이터에는 미포함). ¹

주최측은 모델이 본연의 언어적 패턴과 속성들의 상호작용을 학습하는 대신, 텍스트 내에 우연히 포함된 가격 정보를 정답지처럼 커닝하는 데이터 누수(Data Leakage) 현상을 방지하기 위해 정교한 사전 클렌징을 수행했다.¹ 판매자가 name이나 item_description 텍스트 내에 "\$20"나 "15 dollars"와 같이 실제 가격을 유추할 수 있는 문구를 기재한 경우, 이를 모두 일괄 삭제하고 그 자리를 [rm](Removed의 약자)이라는 특수 태그로 치환하여 제공했다.¹ 이 조치는 모델이 문맥적 가치(Contextual value)를 평가하는 본연의 목표에 집중하도록 유도한다.

3. 평가 지표의 수학적 특성과 타겟 변수의 정규화 메커니즘

예측의 정확도를 측정하고 상금 수상자(1등 \$60,000, 2등 \$30,000, 3등 \$10,000)를 가리기 위한 공식 평가 지표로는 Root Mean Squared Logarithmic Error(RMSLE)가 채택되었다.¹ 단순한 오차의 제곱을 평균 내는 RMSE(Root Mean Squared Error)가 아니라 대상 변수에 자연로그를 취한 뒤 오차를 산출하는 이 지표는, 전자상거래 도메인이 가지는 데이터의 비대칭적 분포를 다루기 위해 수학적으로 치밀하게 고안된 것이다. RMSLE의 산출 수식은 다음과 같다.

$$\epsilon = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(p_i + 1) - \log(a_i + 1))^2}$$

여기서 ϵ 은 최종 산출되는 RMSLE 스코어를 의미하며, n 은 관측치의 총 개수, p_i 는 모델이 예측한 가격, a_i 는 실제 판매된 가격을 나타낸다.¹ 가격 데이터에 1을 더한 후 로그를 취하는 $\log(x + 1)$ 연산은 실제 가격이 0달러인 상황(무료 나눔 등)에서 로그 함수가 마이너스 무한대로 발산하는 것을 방지하는 안전장치이다.¹

RMSLE가 본 문제에 적합한 이유는 크게 세 가지 차원에서 분석될 수 있다.

첫째, 상대적 오차율의 반영이다. 만약 10달러짜리 상품을 20달러로 예측한 경우와 1,000달러짜리 상품을 1,010달러로 예측한 경우, 두 상황 모두 절대적인 오차 금액은 10달러로 동일하다. RMSE 환경에서는 두 모델이 동일한 페널티를 받는다. 그러나 전자상거래 관점에서 전자는 100%의 예측 오차율을 범한 심각한 오류이고, 후자는 단 1%의 오차를 보인 매우 훌륭한 예측이다. 로그를 취해 오차를 계산하면, 값의 스케일이 커지더라도 그 비율적 차이를 공정하게

측정할 수 있다.

둘째, 극단적 이상치(Outlier)에 대한 강건성(Robustness)이다. 수백만 개의 상품 중에는 한정판 시계나 다이아몬드처럼 일반적인 분포를 아득히 벗어나는 초고가 상품들이 존재한다. 이러한 롱테일(Long-tail) 분포 환경에서 로그 변환은 극단적인 값들을 압축시켜, 몇 개의 이상치가 전체 손실 함수(Loss function)의 그래디언트를 독점하는 현상을 차단한다.

셋째, 과소 예측(Under-estimation)에 대한 강력한 페널티 부여다. RMSLE의 수학적 곡선은 실제 가격보다 더 높게 예측했을 때보다, 실제 가격보다 더 낮게 예측했을 때 오차 값이 더 가파르게 상승하는 비대칭성을 지닌다. 이는 플랫폼 비즈니스 관점에서, 판매자에게 너무 낮은 가격을 제안하여 금전적 손실을 입히고 불만을 야기하는 치명적인 상황을 알고리즘 스스로 회피하도록 유도하는 훌륭한 안전망으로 작용한다.

알고리즘 구현체인 main.py는 이러한 평가 지표의 특성을 학습 과정에 온전히 반영하기 위해 훈련 데이터 준비 단계에서 선제적인 전처리를 수행한다. 먼저 판다스(Pandas) 데이터프레임을 통해 데이터를 로드할 때 `train = train[train['price'] > 0]` 구문을 통해 0달러 이하의 비정상적인

거래 기록을 제거한다.¹ 그 후 타겟 변수 `y_train`에 `np.log1p`(자연로그 $\log(x + 1)$)를 적용하여 데이터의 비대칭적 왜곡을 평탄화한다.¹ 여기에 그치지 않고, 변환된 로그 값들의 배열을 다시 Scikit-Learn의 StandardScaler를 통과시켜 평균이 0, 분산이 1이 되도록 스케일링하는 과정을 거친다.¹ 타겟 변수를 표준화하는 이중 전처리는 뒤이어 등장할 신경망 모델의 손실 함수(Mean Squared Error) 최적화 공간을 매끄럽게 만들어 가중치가 발산하지 않고 안정적으로 수렴하도록 돕는 결정적인 역할을 한다.

4. 파편화된 다중 모델에서 단일 모델로의 진화 전략

본 보고서가 분석하는 최종 코드(main.py)의 구조적 우수성을 완벽히 이해하기 위해서는, 이 코드가 탄생하기 전까지 대회 참가자들 사이에서 시도되었던 초기의 복잡한 파이프라인과 그 한계점, 그리고 팀 병합(Team Merge)을 통해 이루어진 패러다임의 전환 과정을 짚고 넘어가야 한다.

4.1. 초기 독립 모델의 한계와 복잡성

대회 중반까지 각 개발자들은 각자의 방법론을 고도화하고 있었다. 한 축을 담당했던 모델 파이프라인(Pawel의 사전 병합 솔루션, 점수 ~0.3950)은 도메인 지식에 기반한 고도의 계층적 분할 전략을 채택했다.¹ 카테고리를 레벨 1, 2, 3으로 쪼개고, 2차 카테고리과 배송비, 상품 상태 등의 조합마다 개별적인 릿지(Ridge) 선형 회귀 모델을 생성하는 파편화된 구조였다.¹ 이러한 수많은 선형 모델들이 1차 예측을 수행하면, 실제 가격과의 잔차(Residual)를 계산하고, 이 잔차를 타겟으로 삼아 희소 데이터를 입력받는 다층 퍼셉트론(MLP)과 LightGBM 앙상블이 남은 오차를 보정하는 부스팅(Boosting) 형태의 3단계 복합 아키텍처가 구축되어 있었다.¹ 이 복잡한 모델은 대략 20분 만에 훈련을 마치고 0.4050의 점수를 냈다.¹ 또 다른 한 축(Konstantin의 사전 병합 솔루션, 점수 ~0.3920)은 딥러닝 자체에 집중했다.¹ Eli5 라이브러리로 릿지 모델을 해석하여 얻은 인사이트를 바탕으로 TensorFlow 기반의 희소 다층 퍼셉트론(Sparse MLP) 모델 3개를 순차적으로 훈련시키고, 이에 더해 Keras 기반의 1D 합성곱 신경망(CNN, Conv1d)을 병행 학습시켰다.¹ CNN 모델 자체의 단독 성능은 다소 떨어졌으나 MLP 모델과 예측 특성이 달라

앙상블 시 점수 향상을 이끌어내는 역할을 했다.¹

4.2. 팀 병합(Merge) 후의 아키텍처 통합 및 의사결정

리더보드 1위와 2위를 다투던 이들은 결국 팀 병합 마감일을 앞두고 힘을 합치기로 결정했다.¹ 일반적인 데이터 과학 대회에서는 두 개의 우수한 솔루션의 예측값을 단순히 산술 평균 내는 것(Blind merge)만으로도 큰 성능 향상을 얻을 수 있지만, 본 대회의 60분 메모리/시간 제약은 그러한 안일한 접근을 불허했다.¹ 기존의 모든 모델을 하나의 커널에 옥여넣으면 필연적으로 메모리 초과 현상이 발생했다.¹

약 2주간의 치열한 최적화 과정(4500개의 Slack 메시지, 152회의 Git 커밋, 31번의 Pull Request 통합, 2015줄의 Python 코드 작성 등 방대한 협업이 동반됨) 끝에 팀은 중대한 사실을 깨닫는다.¹ 카테고리별로 개별 모델을 훈련시키는 것은 겉보기엔 그럴싸하지만 실제로는 딥러닝 모델의 잠재력을 크게 과소평가하는 행위라는 점이다.¹ 신경망은 카테고리 이름, 텍스트 설명, 브랜드 간의 복잡한 비선형적 상호작용을 프로그래머가 강제로 분리하지 않더라도 대규모 데이터 속에서 스스로 상호작용을 포착하여 학습할 수 있는 능력을 지니고 있었다.¹

또한 연산 효율성 측면에서 모델 간의 득실이 확연히 드러났다. 1D 합성곱 신경망(CNN)은 텍스트의 순차적 맥락을 파악하는 데는 좋지만, 임베딩 레이어를 거치며 막대한 행렬 곱셈을 유발했다.¹ CNN 모델 하나가 4개의 코어를 점유하고 임베딩 크기 32로 학습을 완료하는 데 걸리는 동일한 시간 동안, 구조가 단순한 다층 퍼셉트론(MLP)은 은닉층(Hidden Size)을 무려 256까지 키운 상태에서 4개의 독립된 모델을 동시에 학습시킬 수 있었다.¹ 성능 대비 계산 비용을 고려할 때 CNN은 완전히 탈락했다.¹ 이 과정에서 Mixture of Experts (MoE) 같은 첨단 모델 구조도 시도되었으나, TensorFlow 프레임워크가 MoE를 구현하는 데 필요한 일부 희소(Sparse) 텐서 연산을 지원하지 않아 무산되었고, Factorization Machines (FM)과 MLP를 병합하는 이국적인 구조 역시 결국 단순한 MLP 구조에 비해 뚜렷한 이점을 보이지 못했다.¹

결국 이들의 전략은 "단일한 아키텍처(희소 데이터 처리에 최적화된 MLP)만을 사용하되, 입력되는 데이터의 전처리 방식과 훈련 데이터셋을 다변화하여 앙상블의 다양성(Diversity)을 인위적으로 창출하는 것"으로 귀결되었다.¹ 이 결론은 main.py의 핵심 철학을 형성하며, 제한된 시간 내에 여러 모델을 훈련시킬 수 있는 여력을 제공하게 된다.

4.3. 빌드 시스템의 파이프라인 통합

복잡해진 전처리 로직과 여러 모듈로 분산된 코드를 단일 커널에서 실행하기 위해, 이들은 독자적인 파이썬 패키지 빌드 시스템을 구축했다.¹ 전체 소스 코드를 Base64 포맷으로 인코딩하여 거대한 문자열 변수(ENCODED_FILES) 형태로 스크립트 상단에 배치한 후, 실행 시점에 이를 디코딩하여 커널 로컬 환경에 파일을 풀고(DECODE_FILES), 동적으로 파이썬 패키지를 설치한 뒤 메인 함수를 호출하는 기발한 방식을 사용했다.¹ 파이썬 인터프리터가 데이터 전처리 후 발생시키는 고질적인 메모리 누수(Memory Leak) 현상을 방지하기 위해 각 훈련 단계를 완전히 독립된 프로세스로 순차적으로 띄워 실행하는 구조까지 정립했다.¹ (main.py 스크립트는 이 거대한 파이프라인을 하나의 실행 가능한 모듈로 응축한 결과물이다.)¹

5. 텍스트 통합 및 특징 추출 파이프라인의 설계 철학

main.py에 구현된 데이터 전처리 단계(Phase 1 ~ Phase 3)는 정교한 딥러닝 모델로 넘어가기 전, 복잡한 언어 데이터를 가장 가볍고 효율적인 벡터 형태로 압축하는 정제소 역할을 한다.¹ 분석 과정에서 여러 언어학적 특징 공학(Feature Engineering)이 시도되었으나 대부분 폐기되었으며, 극도의 단순화만이 살아남았다.¹

5.1. 텍스트 차원 축소를 위한 컬럼 결합(Text Concatenation)

데이터프레임을 조작하는 preprocess(df) 함수는 판다스 내장 함수를 활용해 놀라운 정도로 직관적인 전처리를 수행한다.¹ 우선 모든 텍스트 컬럼(name, brand_name, item_description, category_name) 내부에 존재하는 결측치(NaN)를 빈 문자열 fillna("")로 일괄 치환한다.¹ 이는 후속 문자열 병합 연산에서 NaN 값이 전파되어 데이터가 유실되거나 타입 오류가 발생하는 것을 차단한다.¹

가장 주목할 만한 조치는 텍스트 필드의 전략적 결합이다.

1. 상품명과 브랜드의 결합: name 컬럼과 brand_name 컬럼을 단일 공백을 두고 하나의 문자열로 결합하여 새로운 name 필드를 덮어쓴다.¹ 상품의 가치는 브랜드의 인지도에 압도적인 영향을 받으므로, 모델이 상품명과 브랜드를 별개의 피처로 학습하는 것보다 연속된 문맥으로 포착하게 만드는 것이 훨씬 유리하기 때문이다.
2. 모든 서술적 정보의 단일화: 그 다음 단계로 item_description, 방금 업데이트된 name, 그리고 category_name을 다시 모두 공백으로 연결하여 하나의 거대한 text 컬럼을 생성한다.¹

이러한 전면적인 텍스트 병합 전략은 과거 연구자들이 여러 컬럼 조합을 테스트한 끝에 도달한 최적해이다.¹ 카테고리과 상품 설명을 별개의 입력 층으로 분리하여 학습시켰을 때보다, 하나로 뭉쳐서 단일한 벡터 공간에 던져 넣었을 때 모델의 검증 점수가 0.37xx 수준으로 도약하는 결정적 계기가 되었다.¹ 신경망은 "나이키"라는 텍스트가 브랜드 컬럼에 있든 카테고리에 있든, 전체 텍스트 내에서 함께 발생하는 동시 출현(Co-occurrence) 패턴을 파악하여 적절한 가격대 가중치를 스스로 형성할 수 있음을 보여준다. 결론적으로 데이터프레임은 복잡한 다중 컬럼 구조에서 벗어나 단 4개의 핵심 컬럼 ['name', 'text', 'shipping', 'item_condition_id']만 남기고 경량화된다.¹

5.2. 폐기된 텍스트 특징 공학(Feature Engineering)과 그 교훈

전처리 과정에서 가장 흥미로운 점은, 적용된 기법보다 '적용되지 않은' 기법들이 주는 시사점이다. 자연어 처리 분야의 전통적인 지침에 따르면 어간 추출(Stemming), 맞춤법 교정(Spell checking) 등을 수행하여 어휘집(Vocabulary)의 크기를 줄이는 것이 상식이다. 실제로 연구자들은 초기에 표준화된 PorterStemmer를 적용해보고, 이름 글자 단위의 N-gram(Chargrams)을 추출하여 데이터 밀도를 높이려는 시도를 했다.¹ 또한 상품 설명 중에 누군가를 위해 특별히 지정된 상품임을 나타내는 "for [이름]"과 같은 문구를 찾아내기 위해 NLTK 라이브러리와 AhoCorasick 문자열 탐색 알고리즘을 사용한 정교한 피처를 생성하기도 했다.¹ 나아가 설명문에 포함된 개행 문자(Newlines)가 단어들을 의도치 않게 이어붙이는 문제를 해결하려 했고, 여러 물품이 묶인 번들(Bundle) 상품에서 "10 data 5 scientists"와 같은 텍스트가 단어 빈도 계산기(CountVectorizer)를 거치면서 'data=10개, scientists=5개'로 숫자가 분리되어

왜곡되는 수치적 벡터화의 오류를 교정하려고 시도했다.¹ 이 수치 교정 아이디어는 단일 데이터셋에서 앙상블 점수를 0.001만큼 소폭 상승시키는 효과를 보이기도 했다.¹

그러나 놀랍게도 최종 `main.py` 통합 스크립트에서는 이러한 모든 인간의 수작업적 개입이 완벽하게 배제되었다.¹ 팀의 핵심 연구자 중 한 명인 Pawel은 이 현상에 대해 "신경망은 마치 '그래, 네가 만든 피쳐 엔지니어링을 써줄게, 여기 보상으로 0.0003점의 성능 향상을 주지'라고 말하는 것 같다"고 회고했다.¹ 즉, 고도로 튜닝된 충분한 크기의 신경망은 굳이 전처리 단계에서 사람이 일일이 문법을 수정하고 형태소를 분석해주지 않아도, 원시 텍스트 자체에 내재된 복잡한 비선형적 상호작용을 직접 포착할 수 있는 딥러닝 고유의 강력한 표현력을 입증한 것이다.¹ 수치 교정 아이디어 역시 추가적인 테스트를 진행할 시간적 여유가 부족하여 최종 파이프라인에서는 채택되지 않았다.¹

5.3. 병렬 처리 기반의 **Feature Union**과 고차원 희소 행렬 변환

전처리를 마친 텍스트와 범주형 데이터는 머신러닝 모델이 이해할 수 있는 숫자의 배열, 즉 벡터(**Vector**)로 변환되어야 한다. `main.py`는 Scikit-Learn의 `make_union` 파이프라인을 구축하여 서로 다른 필드의 벡터화 작업을 물리적으로 동시에 수행하도록 `n_jobs=4`로 설정하여 병렬 처리 효율을 극대화했다.¹

타겟 필드	벡터화 기법	세부 파라미터 및 논리적 배경
name	TfidfVectorizer	• <code>max_features=100000</code>: 출현 빈도가 높은 상위 10만 개 단어만 추출. • <code>token_pattern='w+'</code>: 알파벳, 숫자, 언더바 등 단어 단위 토큰만 추출하여 앞서 언급된 Chargram을 배제하고 처리 속도를 높임.¹
text	TfidfVectorizer	• <code>max_features=100000</code>: 동일하게 10만 차원으로 제한. • <code>token_pattern='w+'</code>: 단어 단위 토큰 사용. • <code>gram_range=(1, 2)</code>: 단일 단어(Unigram)뿐만 아니라 연달아 등장하는 두

		단어(Bigram)의 패턴까지 포착하여 "vintage shirt"와 같은 구(Phrase) 단위의 의미망을 모델에 전달함. ¹
shipping, item_condition_id	DictVectorizer	• to_records 커스텀 함수와 FunctionTransformer를 사용해 행렬을 딕셔너리 리스트로 변환한 후, 원-핫 인코딩 형태의 희소 행렬로 변환.¹

이 과정(Phase 1 ~ Phase 3)을 거쳐 훈련 데이터와 검증 데이터는 20만 차원을 훌쩍 넘는 거대한 수학적 행렬로 탈바꿈한다. 이 행렬의 대부분의 칸은 0으로 채워진 극단적인 희소 행렬(Sparse Matrix) 구조를 띠게 되며, RAM 메모리 용량을 초과하지 않도록 64비트 실수형 대신 float32 타입으로 다운캐스팅(Down-casting)되어 저장된다.¹

6. Keras 3 기반 단일 희소 모델(Sparse Model)의 아키텍처 설계

main.py 내부의 fit_predict 함수는 데이터 변환 이후 본격적인 가격 예측을 수행하는 신경망을 정의하고 훈련시킨다. 앞서 논의했듯, 이 모델 구조는 복잡한 하이브리드 아키텍처를 버리고 오직 다층 퍼셉트론(MLP)의 은닉층을 최대 확장하는 방향으로 설계되었다.¹

6.1. 모델 계층(Layer)의 구조적 특성

이 신경망의 가장 핵심적인 설계 요건은 수십만 차원의 행렬을 밀집(Dense) 행렬로 변환하지 않고 있는 그대로 처리할 수 있어야 한다는 점이다. 만약 20만 차원의 데이터를 밀집 텐서로 변환하는 순간, 16GB의 메모리는 즉각적으로 바닥나고 시스템은 붕괴된다. 따라서 모델의 첫 관문인 입력층은 반드시 Input(sparse=True) 인자를 가져야 하며, 이는 TensorFlow 백엔드에게 이 입력 데이터가 희소성을 띠고 있으므로 메모리 공간을 최적화하여 곱셈 연산을 수행하라는 명시적 지시를 내린다.¹

이후 데이터는 세 개의 완전 연결 계층(Fully Connected Layers)을 순차적으로 통과한다.

- 첫 번째 은닉층: Dense(192, activation='relu')
- 두 번째 은닉층: Dense(64, activation='relu')
- 세 번째 은닉층: Dense(64, activation='relu')
- 출력층: Dense(1)¹

과거 모델 탐색 과정에서 첫 번째 은닉층에 대한 L2 정규화(L2 regularization) 적용이나, TensorFlow 모델에서의 PRELU(Parametric Rectified Linear Unit) 활성화 함수 사용이 RELU보다 약간 더 나은 성능을 낸다는 사실이 확인되기도 했으나¹, 통합된 간결한 main.py 아키텍처에서는 호환성과 훈련 속도를 위해 표준적인 RELU 활성화 함수로 통일되었다.¹ 또한

타겟 가격을 직접 예측하는 대신 가격을 64개의 버킷(Bucket)으로 쪼개고 L2 거리를 계산한 뒤 높은 온도의 Softmax를 적용하여 연성 타겟(Soft Target) 분류 문제로 우회하는 모델이 예측의 과적합(Overfitting)을 방지하는 데 효과가 있었으며¹, 극단적인 가격 이상치에 대한 민감도를 줄이기 위해 Huber Loss(후버 손실)를 도입한 변형 모델도 앙상블의 다양성을 더해주었다.¹ 그러나 스크립트를 하나로 압축하고 Keras 최신 버전에 맞게 이식하는 과정(main.py)에서는 이러한 부가적인 변형들을 걷어내고 가장 직접적인 회귀 손실 함수인 평균 제곱 오차(mean_squared_error)와 Adam 옵티마이저(학습률 3e-3)를 채택하여 연산의 직관성을 극대화했다.¹ 출력층에는 가격이라는 단일 연속형 수치를 도출해야 하므로 활성화 함수가 생략된 단일 노드(Dense(1))가 배치된다.¹

6.2. 프레임워크 마이그레이션과 상태 격리(State Isolation) 메커니즘

main.py는 과거 TensorFlow 1(TF1) 기반의 코드에서 TensorFlow 2(TF2) 및 Keras 3 환경으로 전면 마이그레이션된 형태를 띠고 있다.¹ 구형 코드에서 필수적으로 사용되던 tf.Session 생성이나 ks.backend.set_session과 같은 복잡한 그래프 컨텍스트 관리 코드가 모두 제거되었다.¹

가장 중요한 기술적 개선점은 Keras 3가 제공하는 자동화된 그래프 격리 기능이다. 다수의 모델을 반복적으로 생성하고 훈련시키는 커널 환경에서는 객체가 제대로 파괴되지 않아 메모리가 누수되는 현상이 잦다. 그러나 Keras 3 환경에서는 fit_predict 함수가 호출될 때마다 프레임워크가 백그라운드에서 완전히 새롭고 깨끗한 모델 그래프를 독립적인 메모리 공간에 동적으로 생성한다.¹ 이를 통해 이전 모델의 가중치나 그라디언트 찌꺼기가 다음 모델로 넘어가는 것을 완벽하게 차단하며, 60분 내내 안정적인 메모리 점유율을 유지할 수 있게 된다. 시간에 민감한 루틴들을 추적하기 위해 timer(name)라는 사용자 정의 컨텍스트 매니저를 도입하여 파이프라인의 각 단계(Phase)별 병목 구간을 즉각적으로 모니터링할 수 있도록 조치한 점도 실용적인 공학의 산물이다.¹

7. 훈련 속도 최적화 메커니즘: 동적 배치 크기 조절 (Dynamic Batch Sizing)

복잡도 높은 MLP 모델이 4개의 코어만 존재하는 열악한 CPU 환경에서 350만 건에 달하는 Stage 2 훈련 데이터를 제한 시간 내에 학습하기 위해 main.py에 도입된 가장 돋보이는 최적화 전략은 바로 학습 스케줄의 튜닝, 즉 에폭(EPOCH)이 진행됨에 따라 배치 크기(Batch Size)를 기하급수적으로 확장하는 기법이다.¹

일반적으로 신경망 학습에서는 고정된 미니 배치 크기를 사용한다. 그러나 본 솔루션에서 모델은 전체 데이터셋을 단 3번만 순회(3 Epochs)하며, 각 단계마다 훈련 데이터 뭉치의 크기를 다음과 같이 2배씩 늘려나간다.

학습 진행 단계	적용된 배치 크기 (Batch Size)	논리적 목적 및 모델 역학
1	16	초기 학습 안정화
2	32	학습 속도 향상
3	64	최종 성능 극대화

Epoch 1	2^{11} (2048) ¹	비교적 작은 배치 크기를 사용하여 기울기(Gradient) 업데이트에 무작위성(Noise)을 허용한다. 이를 통해 모델이 훈련 초기 국소 최적점(Local Minima)에 갇히는 것을 방지하고 거대한 특성 공간을 넓고 빠르게 탐색하도록 유도한다. ¹
Epoch 2	2^{12} (4096) ¹	배치 크기를 늘려 기울기 추정치의 분산을 줄이고, 더 정확하고 안정적인 방향으로 가중치 업데이트가 이루어지도록 유도한다. 통상적으로 이 시점에서 검증 점수(Validation Score)가 최고점에 도달하게끔 하이퍼파라미터가 튜닝되었다. ¹
Epoch 3	2^{13} (8192) ¹	압도적인 배치 크기를 통해 훈련 속도를 극한으로 끌어올린다. 연구진은 이 3번째 에폭에서 의도적으로 모델을 훈련 데이터에 과적합(Overfit)시키도록 유도했다. 개별 모델이 약간 과적합되더라도 앙상블 과정에서 모델 간의 예측 다양성이 강력한 시너지를 내어 최종 검증 오차가 오히려 감소하기 때문이다. ¹

이러한 동적 배치 스케줄링의 진정한 파괴력은 하드웨어 병목의 해소에 있다. 거대한 희소 행렬을 CPU로 연산할 때, 배치가 작으면 연산 자체보다 가중치를 업데이트하고 메모리를 동기화하는 빈도가 너무 잦아져 막대한 실행 시간(Wall-clock time) 지연이 발생한다. 매 에폭마다 배치 크기를 두 배로 확장함으로써, 훈련 후반부로 갈수록 모델 훈련 속도가 비약적으로 빨라진다.¹ 연구진은 초기 단계에서 학습률(Learning rate)을 점진적으로 낮추는 동시에 배치 크기를 늘리는 실험을 진행하여 성능을 고도화했으며¹, main.py에서는 이러한

철학을 계승하여 하드웨어 제약을 정면으로 돌파하는 데 성공했다.¹

8. 동시성 제어 한계 극복: 다중 스레드 억제 및 병렬 프로세싱 매핑

데이터 증폭기(Stage 2)가 가동되었을 때, 이 알고리즘이 시스템 다운 없이 끝까지 생존할 수 있었던 이유는 내부적으로 운영체제와 머신러닝 프레임워크 간의 스레드 경합(Thread Contention)을 완벽히 통제했기 때문이다.

Kaggle 커널이 제공하는 4 코어 CPU 환경에서 TensorFlow와 같은 고성능 수학 연산 라이브러리들은 본능적으로 모든 가용 코어를 점유하기 위해 수많은 스레드를 파생(Spawning)시킨다.¹ 만약 데이터가 조밀한 밀집(Dense) 형태라면 이러한 내부 멀티스레딩이 연산을 가속하겠지만, 20만 차원에 육박하는 희소(Sparse) 텐서를 다룰 때는 정반대의 현상이 발생한다. 스레드들이 등성등성 존재하는 0이 아닌 값들을 찾아 메모리를 무작위로 접근하면서 코어 간의 캐시 일관성을 유지하기 위해 끊임없이 락(Lock)을 걸고 해제하는 오버헤드가 발생하여 선형적인 속도 향상(Linear Scaling)이 전혀 이루어지지 않는 것이다.¹

과거 연구 과정에서 이를 회피하기 위해 MXNet 프레임워크의 도입이 시도되기도 했다. MXNet은 희소 다층 퍼셉트론에 대한 CPU 연산 효율이 매우 뛰어나 초기 파이썬 버전에서는 TensorFlow보다 2배나 빠른 훈련 속도를 보여주었으며, 더 작은 배치 크기에서도 속도 저하가 없었다.¹ 그러나 결정적으로 MXNet의 실행 엔진은 스레드 안전성(Thread-safe)을 보장하지 못했다.¹ 스레딩을 이용해 병렬화를 시도하면 단 1개의 코어만 활성화되거나 치명적인 메모리 접근 오류(Segfault)가 발생하여 프로그램이 강제 종료되었다.¹ 프로세스 기반의 다중 처리(Multiprocessing)를 사용하고 데이터를 공유 메모리에 올리려는 시도 역시 제공된 1GB 남짓의 디스크/메모리 한계선에 부딪혀 결국 폐기 처분되었다.¹ 결국 12개의 모델을 훈련시키기 위해 MXNet 기반 솔루션과 TensorFlow 기반 솔루션을 동시에 분리하여 제출하는 우회로를 택해야만 했다 (당시 MXNet Public 0.37665, TF Public 0.37920 기록).¹

이러한 치열한 시행착오의 결과로, 통합된 main.py 스크립트는 TensorFlow 환경에 잔류하되 프레임워크의 고질적인 다중 스레딩을 강제로 억압하는 비범한 전역 스레드 격리(Global Threading Isolation) 전략을 구사한다.

1. 운영체제 레벨의 차단: 스크립트 실행의 가장 첫 줄에서 환경 변수 `os.environ = '1'`을 삽입하여, 하부 C/C++ 라이브러리의 OpenMP 멀티 스레딩을 1개로 제한한다.¹
2. 프레임워크 내부 스케줄러 제한: main 함수 내부 진입 시 TensorFlow의 병렬 처리 설정을 직접 제어한다.
 - `tf.config.threading.set_intra_op_parallelism_threads(1)`: 단일 행렬 곱 연산 등을 수행할 때 코어를 쪼개어 쓰지 못하게 막는다.¹
 - `tf.config.threading.set_inter_op_parallelism_threads(1)`: 서로 독립적인 여러 연산을 동시에 처리하려는 내부 스케줄러의 행위를 금지한다.¹ (과거 비공식적으로 `use_per_session_threads=1` 옵션을 써서 프로세스를 늘리지 않고 메모리를 아끼던 기법이 현대적 문법으로 진화한 형태이다¹).

이렇게 각 신경망 모델이 오직 1개의 논리적 코어만 사용하도록 완전히 격리시킨 후, 파이썬의 `multiprocessing.dummy.ThreadPool(processes=4)`를 호출한다.¹ 이 스레드 풀은 4개의 독립적인 모델 훈련 작업(`fit_predict`)을 커널이 가진 4개의 물리적 코어에 1:1로 매핑하여 동시에 실행시킨다.¹ 결과적으로 프레임워크 내부의 스레드 충돌 오버헤드는 0으로 수렴하면서도 커널 하드웨어의 연산 잠재력은 100% 한계치까지 활용하는 가장 우아한 병렬 아키텍처가 완성된 것이다.

9. 특징 이진화(Feature Binarization)와 가중치 평균화(Ensemble) 전략

단일한 MLP 아키텍처만으로 높은 리더보드 점수를 쟁취하기 위해서는 복수의 모델이 만들어내는 예측값들이 통계적으로 유의미한 다양성(Diversity)을 확보해야 한다. 모든 모델이 똑같은 데이터를 학습하여 똑같은 실수를 반복한다면 앙상블은 무용지물이다. `main.py`의 파이프라인(Phase 4 및 Phase 5)은 모델의 구조를 바꾸는 대신 데이터의 형태를 인위적으로 비틀어 모델들의 관점을 다변화하는 훌륭한 창의성을 보여준다.

9.1. 데이터 증강의 마법: TF-IDF에서 Boolean으로의 변환

Phase 4에서 스레드 풀에 작업을 분배하기 직전, 기존의 TF-IDF 파이프라인을 거쳐 완성된 실수형(Float32) 행렬 `X_train`과 `X_valid`를 그대로 복제하여 이진화(Binarization)된 새로운 데이터셋 `Xb_train`과 `Xb_valid`를 창조한다.¹ 변환 로직은 극도로 단순하다. 데이터에 존재하는 모든 0보다 큰 실수 값을 불리언(`np.bool`) 타입, 즉 `True(1)`로 캐스팅한 다음, 신경망이 소화할 수 있도록 다시 `np.float32`로 되돌려 놓는 것이다.¹

이 사소해 보이는 변환(`X.astype(np.bool).astype(np.float32)`)의 파급 효과는 실로 막대하다. 기존의 TF-IDF 행렬은 단어의 빈도와 희귀성을 바탕으로 "이 단어가 해당 상품에서 상대적으로 얼마나 중요한가?"를 연속적인 가중치로 모델에게 알려준다. 반면 이진화된 행렬은 가중치의 강약을 모두 무시하고, 오로지 "이 단어가 존재하는가, 존재하지 않는가?"라는 절대적인 이진 논리만을 제공한다. 이는 추가적인 연산 비용을 들이지 않고도 마치 텍스트 원본에 `CountVectorizer(binary=True)`를 통과시켜 생성한 완전히 새로운 독립 데이터셋을 무료로 얻어낸 것과 완벽히 동일한 효과를 발휘한다.¹ 개발자 Pawel의 아이디어였던 이 기법은 극단적인 빈도수로 인해 신경망의 특정 가중치가 과도하게 증폭되는 현상을 억제하고 앙상블의 강건성을 부여하는 신의 한 수로 작용했다.¹ (0이 아닌 임계값을 설정하여 이진화하려는 후속 실험도 있었으나 큰 개선이 없어 가장 단순한 형태가 유지되었다¹).

9.2. K-Fold 검증 분할 및 병렬 학습 지시

전체 훈련 데이터는 Scikit-Learn의 `KFold(n_splits=20)`를 통해 20개의 조각으로 분할되며, 그중 첫 번째 분할을 취하여 95%의 훈련셋과 5%의 검증셋으로 나눈다.¹ 타겟 변수 `y_train`은 모든 모델에 공통으로 들어가도록 `partial` 함수로 묶어 고정해 둔다.¹ 이후 스레드 풀은 `[[Xb_train, Xb_valid], [X_train, X_valid]] * 2` 형태의 중첩 리스트 구조를 맵핑(`map`)한다.¹ 즉, 1번과 2번 코어에는 이진화된 `Xb` 데이터셋을 입력받는 모델 2개가 배치되고, 3번과 4번 코어에는 본래의 X TF-IDF 데이터셋을 입력받는 모델 2개가 배치되어 각자 독립적인 가중치 초기화와 데이터

섭취를 시작한다.¹ 4개의 모델은 구조가 완벽히 동일하지만, 데이터의 밀도(Density) 차이로 인해 각기 다른 관점의 특징 상호작용을 포착하게 된다.

9.3. 최종 앙상블 및 타겟 복원 메커니즘

60분의 카운트다운이 진행되는 동안 개별 모델의 훈련이 종료되면, 각 모델이 검증 데이터에 대해 내뱉은 4개의 연속형 예측값 배열 리스트(preds_list)가 수집된다.¹ main.py의 최종 Phase 5 단계에서는 이 4개의 예측 텐서를 축 기준으로 단순 병합한 뒤 np.mean(preds_list, axis=0)을 적용하여 산술 평균을 도출한다.¹

과거 연구 단계에서는 12개의 모델에서 나온 예측값을 병합하기 위해 1%의 별도 검증 데이터(로컬 환경에서는 5%)를 구축하고 올가미(Lasso) 회귀 모델을 적용하여 블렌딩 가중치를 정밀하게 튜닝하려는 시도가 있었다.¹ 그러나 L1 정규화를 수반하는 Lasso는 본질적으로 특성 선택(Feature Selection)을 강제하는 성향이 강해, 연구진이 실험적으로 앙상블에 기여할 것이라 믿고 추가한 새로운 모델들의 예측 가중치를 무자비하게 0으로 만들어 버리는 아이러니한 상황을 연출했다.¹ 이러한 시행착오를 거쳐 가장 직관적이고 과적합 위험이 적은 '단순 평균' 방식이 코드 통합 과정에서 정착된 것이다.¹

평균화가 완료된 값은 그 자체로는 의미가 없다. 앞서 Phase 3에서 price 변수에 적용했던 왜곡을 원래대로 되돌려야 한다. 표준화 스케일러 y_scaler의 역변환 함수(inverse_transform)를 호출하여 분산과 평균을 원래 로그 분포 스케일로 돌려놓고, 여기에 지수 함수

np.exp(m1(자연상수 $e^x - 1$))을 적용함으로써 자연로그 log1p 처리가 진행되기 이전의 순수한 실제 달러(USD) 가격 추정치로 완벽하게 복원시킨다.¹ 복원된 이 예측값들은 최종적으로 원래의 판매 가격 타겟 변수와 비교되며, Scikit-Learn 라이브러리의 mean_squared_log_error 함수를 통과하여 최종 모델의 RMSLE 예측 스코어를 화면에 출력하게 된다.¹

10. 종합 시사점

Mercari 가격 제안 알고리즘 대회는 대규모의 비정형 텍스트(상품명, 설명문)와 정형 메타데이터(카테고리, 배송비 등)가 결합된 환경에서 극단적인 비대칭 분포를 지닌 타겟 변수를 회귀 추정해야 하는 고난도의 기계학습 과제였다. 그러나 본 문제의 진정한 본질적 난이도는 알고리즘의 복잡성이 아니라, "Kernels Only"라는 인프라적 감옥, 즉 Stage 2에서 350만 건으로 5배 폭발하는 데이터를 단 16GB 메모리와 60분 이내에 뚫어내야 한다는 잔혹한 엔지니어링 제약 조건에 있었다.

제시된 main.py 통합 솔루션은 이러한 극단적 제약을 극복하기 위해 소프트웨어 공학의 기교와 수학적 통찰을 결합한 탁월한 마스터피스이다. 이 코드가 성공할 수 있었던 결정적인 이유는 '딥러닝 모델의 복잡도'를 버리고 '인프라 최적화와 데이터 다변화'에 사활을 걸었기 때문이다. 모델의 첫 관문에서 복잡한 자연어 처리와 형태소 분석을 모두 기각하고, 모든 텍스트를 공백으로 단순 결합하여 lw+ 단위의 Feature Union 병렬 토큰화에 의존한 것은 데이터 처리의 견고함을 극대화했다.

또한 운영체제 레벨의 OpenMP 설정 차단부터 TensorFlow의 내부 스레딩 속성 강제 고정, 멀티프로세싱 스레드 풀의 1:1 코어 매핑에 이르는 집요한 동시성 제어는 수십만 차원의 희소 다층 퍼셉트론(Sparse MLP) 행렬 연산에서 발생하는 컨텍스트 스위칭의 병목을 완벽히

제거했다. 에폭을 거듭할 때마다 2048에서 8192로 급격히 증가하는 동적 배치 사이즈(Dynamic Batch Sizing) 스케줄링은 로컬 미니마를 회피하는 동시에 실행 시간을 극적으로 압축해 주었다.

가장 돋보이는 통찰은 자원 소모를 수반하는 다중 아키텍처 앙상블을 포기하고, TF-IDF의 부동소수점을 단순 부울(Boolean) 값으로 이진화(Binarization) 재캐스팅하여 마치 CountVectorizer 모델을 하나 더 훈련한 것과 같은 착시를 주어 평균 앙상블의 다양성을 창출한 부분이다. 결론적으로 이 알고리즘 파이프라인 구조는 대규모 텍스트 트래픽과 실시간 추론 제약이 존재하는 실제 전자상거래 인프라스트럭처에서 추천 시스템 및 가격 예측 시스템을 설계할 때 가장 우선적으로 고려해야 할 가볍고 안정적인 기준선(Baseline)을 완벽하게 제시하고 있다.

현재 분석된 main.py 코드는 60분이라는 극한의 실행 시간과 16GB 메모리 제약을 극복하기 위해 극도로 단순화된 휴리스틱(단순 산술 평균 앙상블, 빈도수 기반 특징 추출 등)을 채택하고 있습니다. 하지만 제약 조건이 완화되거나 성능의 극대화가 필요한 상황이라면, 오퍼레이션 리서치(OR)의 수리적 최적화(Mathematical Optimization) 기법과 솔버(Solver)를 현재 코드에 적용하여 알고리즘을 고도화할 수 있는 강력한 여지가 존재합니다.

크게 다음 세 가지 관점에서 최적화 기법을 적용할 수 있습니다.

1. 앙상블 가중치 최적화 (2차 계획법, Quadratic Programming)

현재 main.py 코드는 4개의 독립적인 모델에서 산출된 예측값을 축 기준으로 단순 평균(np.mean)하여 최종 예측값을 도출합니다. 하지만 개별 모델이 가진 편향과 분산이 각기 다르기 때문에 수리적 최적화를 통해 오차를 최소화하는 최적의 가중치를 찾을 수 있습니다.

이를 가중치의 합이 1이 되고 각 가중치가 0 이상이라는 제약 조건 하에서, 검증 데이터의 평균 제곱 오차(MSE)를 목적 함수로 하여 최소화하는 2차 계획법(Quadratic Programming) 문제로 공식화할 수 있습니다. SciPy 라이브러리의 SLSQP(Sequential Least Squares Programming) 알고리즘이나 CVXPY와 같은 볼록 최적화 솔버를 도입하면 최적의 블렌딩 가중치를 수학적으로 도출하여 예측 성능을 끌어올릴 수 있습니다.

2. 변수 선택 (혼합 정수 계획법, Mixed Integer Programming)

현재 코드의 데이터 전처리 단계에서는 TfidfVectorizer의 max_features=100000 파라미터를 사용하여 단순히 단어 출현 빈도순으로 상위 10만 개의 특성만 걸러내어 사용하고 있습니다.

빈도수가 아닌 실제 타겟 가격 예측에 기여하는 중요도를 기준으로 변수를 선택하기 위해 혼합 정수 최적화(MIP)를 적용할 수 있습니다. 변수의 선택 여부를 0 또는 1의 이진 정수 변수로 제한하는 \$L_0\$ 제약을 목적 함수에 포함시킨 뒤 솔버를 가동하면, 다중공선성을 유발하거나 불필요한 노이즈 특징들을 수학적으로 제거하여 최적의 변수 부분집합(Best Subset)을 도출하는 정교한 특징 선택(Feature Selection)이 가능해집니다.

3. 앙상블 가지치기 (정수 계획법, Integer Programming)

대회 당시 자원 제약으로 인해 최종 코드는 4개의 모델만 훈련시켰지만, 오프라인 환경에서 수십 개의 다채로운 모델 풀(Pool)을 생성할 여력이 있다면 앙상블 가지치기(Ensemble Pruning)

기법을 도입할 수 있습니다.

무수히 많은 분류기(Classifier)나 회귀 모델을 전부 사용하는 대신, 정수 계획법(Integer Programming)을 통해 앙상블 내부의 다양성(Diversity)을 최대화하면서도 예측 성능이 가장 높은 최적의 모델 부분집합만 선택하도록 솔버에 위임하는 방식입니다. 이를 통해 60분이라는 제한된 추론 시간 내에 가벼우면서도 가장 강력한 예측 조합을 동적으로 구성할 수 있습니다.

결론적으로, 현재 단순 병합에 의존하는 코드의 마지막 추론 단계 및 특징 선택 단계를 수리적 목적 함수와 제약 조건으로 치환하여 전문 솔버(CPLEX, Gurobi, CVXPY 등)와 결합한다면, 머신러닝의 경험적 한계를 오퍼레이션 리서치(OR)의 수학적 최적해로 극복하는 훌륭한 연구 확장이 될 수 있습니다.

참고 자료

1. 문제 설명.txt